In God We Trust

عنوان: پروژه اول درس شبکه

اسامی دانشجویان: فاطمه بلوک‌آذری – مهدیس میرزایی

شماره دانشجویی: 810102605 - 810102550

بهار ۱۴۰۵

توضیح معماری

در معماری این پروژه یک برنامه به عنوان سرور اجرا می‌شود و چندین کلاینت می‌توانند به صورت همزمان به آن متصل شده و با یکدیگر تعامل داشته باشند.

فایل سرور :

در سمت سرور ابتدا یک سوکت از نوع TCP با استفاده از تابع socket ایجاد می‌شود.

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);
```

برای اینکه در صورت ری‌استارت شدن سرور بتوان دوباره از همان پورت استفاده کرد و خطای Address already in use رخ ندهد، از تابع setsockopt استفاده می‌کنیم.

```
int opt = 1;
if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) < 0)
{
    perror("setsockopt failed");
    exit(EXIT_FAILURE);
}
```

سپس با استفاده از تابع bind این سوکت به یک آدرس IP و پورت 8080 متصل می‌شود تا سرور بتواند درخواست‌های ورودی را دریافت کند.

```
sockaddr_in server_addr;
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(8080);
server_addr.sin_addr.s_addr = INADDR_ANY;

if (::bind(server_fd, (sockaddr *)&server_addr, sizeof(server_addr)) < 0)
{
    perror("Bind failed");
    return 1;
}
```

پس از آن سرور با فراخوانی تابع listen در وضعیت انتظار برای اتصال کلاینت‌ها قرار می‌گیرد.

```
if (listen(server_fd, 10) < 0)
{
    perror("Listen failed");
    return 1;
}
cout << "Server started...\n";
```

سپس برای فایل ها یک فولدر ایجاد میکنیم:

```
filesystem::create_directory(SERVER_FILES_DIR);
```

و در نهایت در این قسمت یک حلقه while داریم که مداوم اجرا می‌شود تا بتواند کلاینت‌های جدید را دریافت کند. ابتدا متغیرهایی برای ذخیرهٔ آدرس و اندازهٔ آدرس کلاینت تعریف می‌شوند. سپس با استفاده از تابع accept، سرور منتظر اتصال یک کلاینت می‌ماند و پس از برقراری اتصال، یک سوکت جدید برای ارتباط با آن کلاینت ایجاد می‌شود. همچنین یک thread جدید برای تابع handle_client ایجاد می‌شود و با دستور detach میتوان چندین کلاینت به طور همزمان را هندل کرد.

```
while (true)
{

    sockaddr_in client_addr;
    socklen_t client_size = sizeof(client_addr);

    int client_socket = accept(server_fd,
                               (sockaddr *)&client_addr,
                               &client_size);

    cout << "New client connected\n";

    thread t(handle_client, client_socket);
    t.detach();
}
```

فایل کلاینت :

ابتدا یک سوکت TCP ساخته می‌شود. و سپس آدرس سرور تنظیم می‌شود و سپس با connect تلاش می‌شود به سرور وصل شود.

```
int sock = socket(AF_INET, SOCK_STREAM, 0);

sockaddr_in server_addr;
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(8080);
server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

if (connect(sock, (sockaddr *)&server_addr, sizeof(server_addr)) < 0)
{
    perror("Connection failed");
    return 1;
}
cout << "Connected to server\n";
```

سپس یک thread ساخته می‌شود که تابع receive_messages را اجرا می‌کند تا پیام‌های سرور را دریافت کند. با detach این thread مستقل اجرا می‌شود و تا بدین صورت برنامهٔ اصلی بتواند همزمان ورودی کاربر را بگیرد.

```
thread t(receive_messages, sock);
t.detach();
```

سپس کلاینت باید username خود را وارد کند و این اولین پیامی است که پس از اتصال برای او نمایش داده می‌شود. پس از آن هر پیامی که می‌دهد با دستور send از طریق سوکت برای سرور ارسال می‌شود و صرفا اگر دستور GET یا PUT باشد مسیر پوشهٔ فعلی (current_path) به انتهای پیام اضافه می‌شود.

```
    cout << "Please enter your username\n";
    string input;
    bool first_message = true;

    while (true)
    {
        getline(cin, input);
        if (first_message)
        {
            input = "NAME " + input;
            first_message = false;
        }
        else
        {
            string working_dir = filesystem::current_path().string();
            int first_space = input.find(' ');
            string command_name = input.substr(0, first_space);
            if (command_name == "GET" || command_name == "PUT")
                input += " " + working_dir;
        }
        send(sock, input.c_str(), input.size(), 0);
    }
```

همچنین تابع receive_messages برای دریافت پیام‌های سرور استفاده می‌شود. یک بافر ۱۰۲۴ بایتی ساخته می‌شود و در یک حلقهٔ بی‌نهایت با recv داده‌ها از سوکت خوانده می‌شوند. اگر تعداد بایت‌های دریافتی کمتر مساوی صفرتا باشد یعنی اتصال قطع شده و حلقه تمام می‌شود. در غیر این صورت پیام دریافتی در خروجی نمایش داده می‌شود.

```
void receive_messages(int sock)
{
    char buffer[1024];
    while (true)
    {
        memset(buffer, 0, sizeof(buffer));
        int bytes = recv(sock, buffer, sizeof(buffer), 0);
        if (bytes <= 0)
        {
            cout << "Disconnected from server\n";
            break;
        }
        cout << buffer << endl;
    }
}
```

حال هر پیامی که از سمت کلاینت ارسال می‌شود توسط تابع recv دریافت شده و داخل buffer ذخیره می‌شود. اگر مقدار bytes کمتر یا مساوی صفر باشد یعنی اتصال کلاینت قطع شده است؛ در این حالت سوکت با close بسته می‌شود و کلاینت از لیست clients حذف شده و حلقه پایان می‌یابد. در غیر این صورت، محتوای buffer به یک string تبدیل می‌شود و در متغیر command قرار می‌گیرد. سپس این رشته به تابع handle_command ارسال می‌شود تا دستور ارسال‌شده توسط کلاینت پردازش شود.

```
void handle_command(int client_socket, string command)
{
    int first_space = command.find(' ');
    string command_name = command.substr(0, first_space);
    string rest = (first_space != command.length()) ? command.substr(first_space + 1) : "";
    if (command_name == NAME_COMMAND)
    {
        handle_name(client_socket, rest);
    }
    else if (command_name == MSG_COMMAND)
    {
        handle_msg(client_socket, rest);
    }
    else if (command_name == USERS_COMMAND)
    {
```

سپس به ازای هر یک از دستورها یک تابع وجود دارد. همچنین دستور اول که کاربر username خود را وارد میکند به صورت دستور name در نظر گرفته شده است.

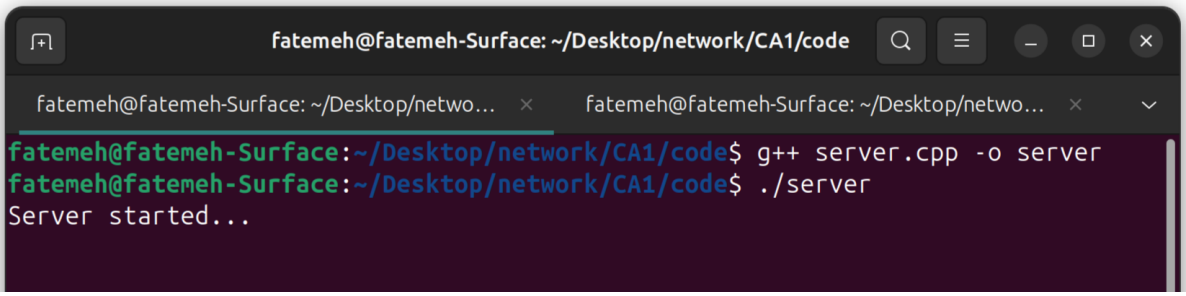
همچنین یک تابع به عنوان broadcast وجود دارد که به تمام کلاینت‌ها غیر از آن کلاینتی که می‌خواهیم پیامی را ارسال میکند. تابع message_to_client نیز با دریافت client_socket و یک رشته, آن را به کلاینت مربوطه با استفاده از تابع send ارسال میکند.

```
void broadcast(string msg, int sender)
{
    for (auto &c : clients)
    {
        if (c.first != sender)
        {
            send(c.first, msg.c_str(), msg.size(), 0);
        }
    }
}

void message_to_client(int client_socket, string message)
{
    send(client_socket, message.c_str(), message.size(), 0);
}
```

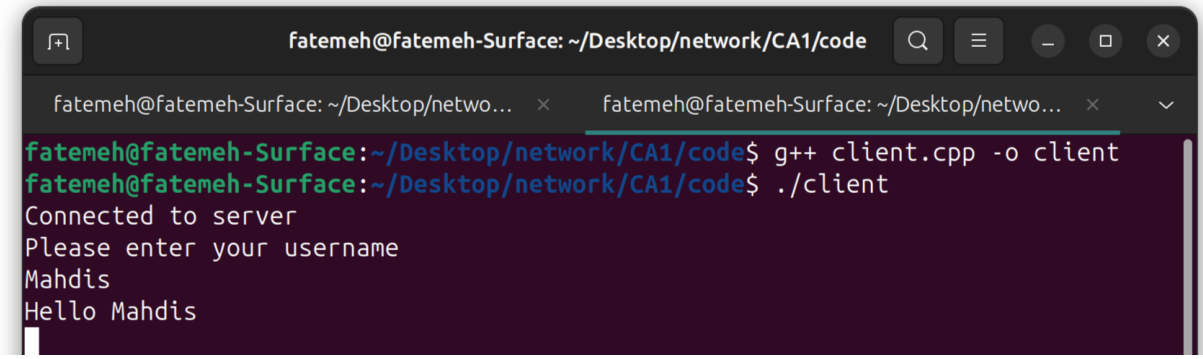
دستورات

ابتدا باید برنامه server اجرا شود.

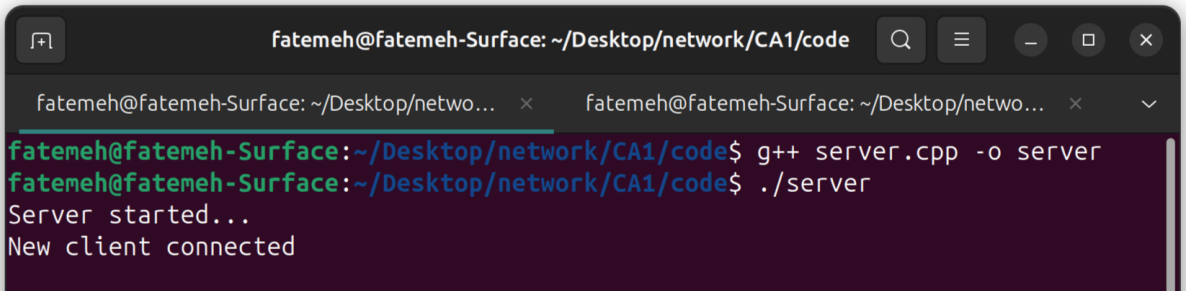


سپس وقتی client جدید اجرا می‌شود، از او خواسته می‌شود تا نام خود را وارد کند و پس از آن به او پیغام سلام داده می‌شود. این کار در تابع زیر رخ می‌دهد.

```
void handle_name(int client_socket, string username)
{
    clients[client_socket] = username;
    message_to_client(client_socket, "Hello " + username);
}
```

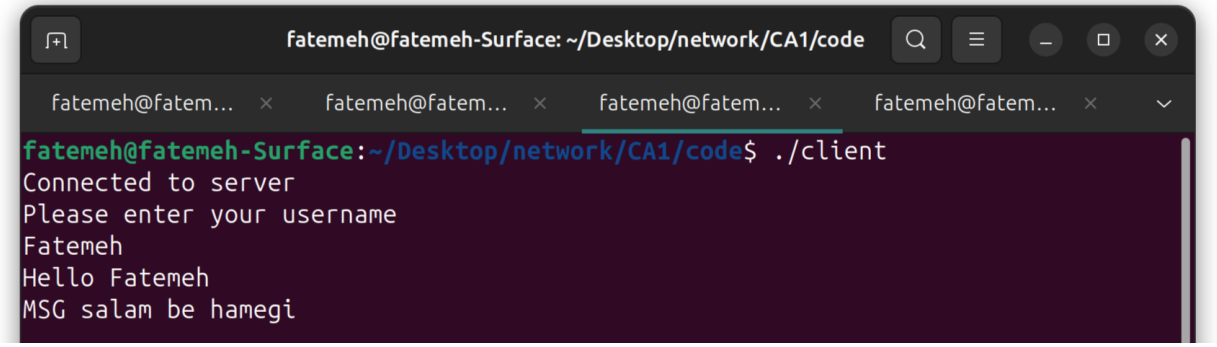


همچنین سرور تشخیص می‌دهد که client جدید متصل شده است.



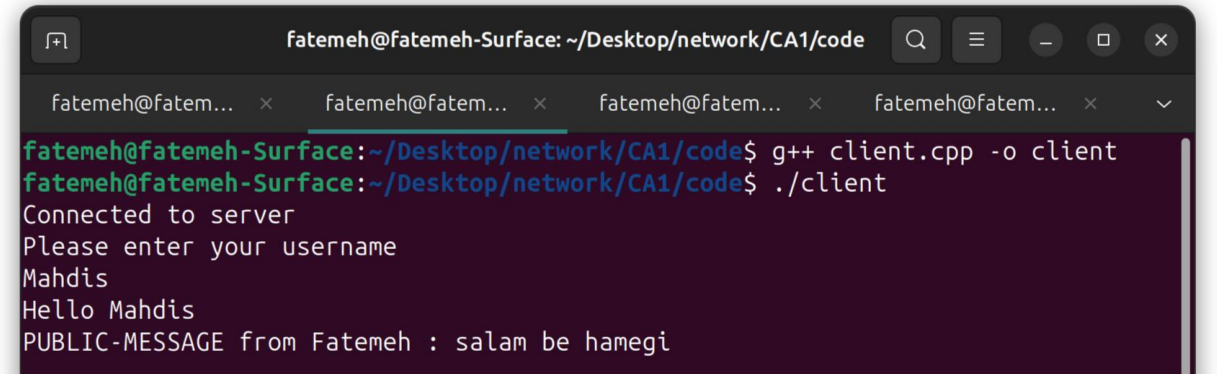
برای ادامه توضیحات، ۳ client مختلف به نام‌های Mahdis, Fatemeh, Marziyeh تشکیل داده‌ایم تا دستورات بعدی به طور واضح‌تری بیان شوند.

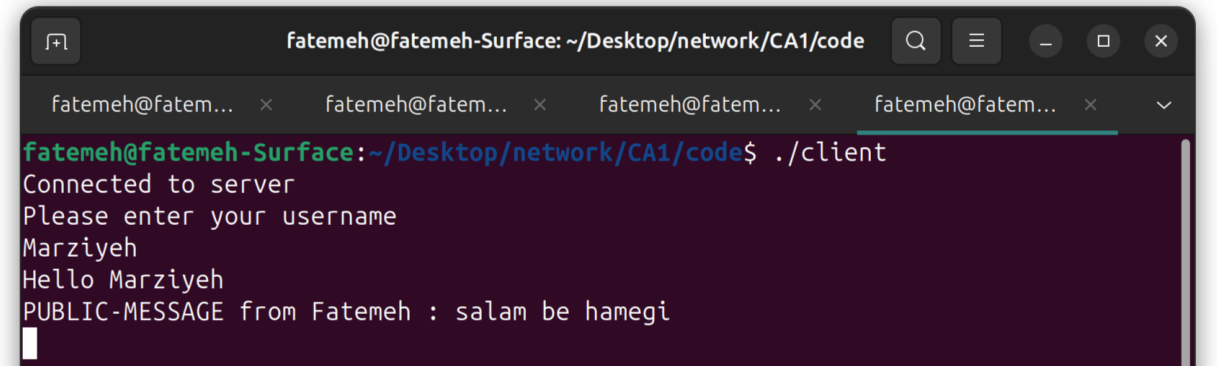
MSG (Message)



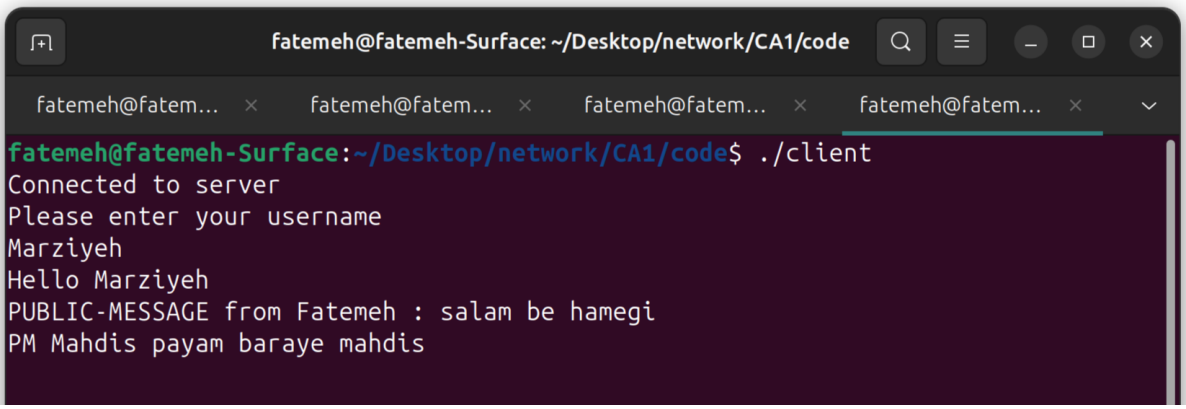
به این صورت Fatemeh پیام را برای همه ارسال می‌کند و همه به جز خود او، پیام را دریافت می‌کنند. همانطور که در تابع زیر مشخص است پیامی به سایر کلاینت‌ها با استفاده از تابع کمکی broadcast ارسال می‌گردد.

```
void handle_msg(int client_socket, string message)
{
    string pub_message = "PUBLIC-MESSAGE from " + clients[client_socket] + " : " + message;
    broadcast(pub_message, client_socket);
}
```



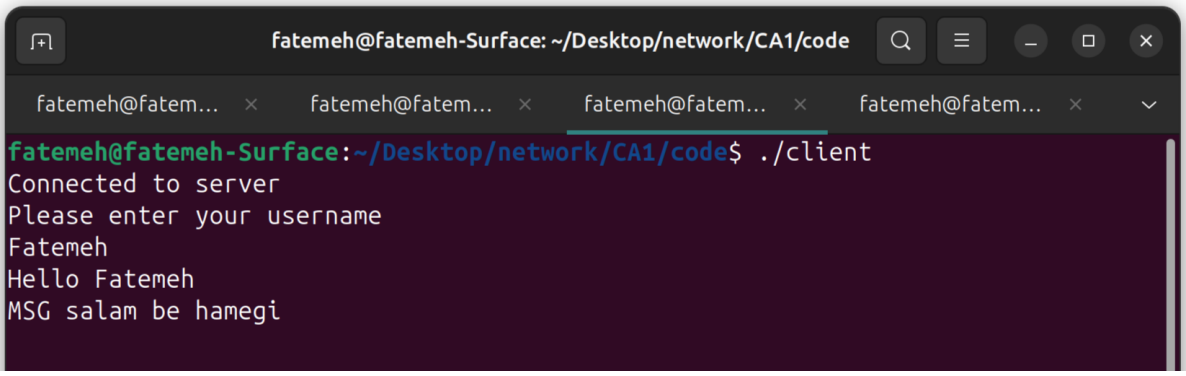


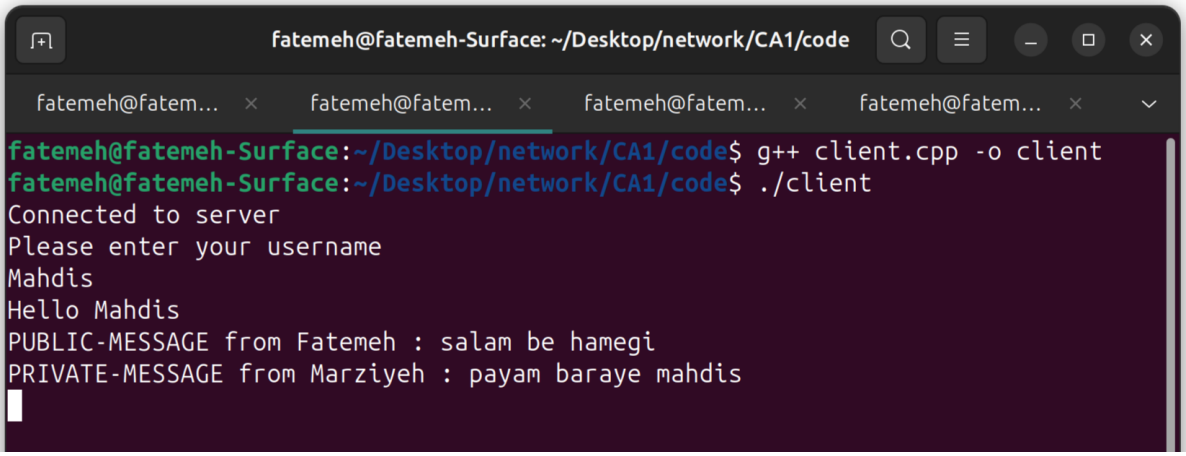
PM (Private Message)



پیام شخصی توسط Marziyeh به Mahdis ارسال می‌شود و Mahdis پیام را دریافت کرده اما Fatemeh دریافت نمی‌کند. همانطور که در تابع زیر مشخص است از بین کلاینت‌ها ارسال آن کلاینتی که user به آن می‌خواهد پیام دهد پیدا می‌شود. در صورتیکه چنین کلاینتی پیدا نشد به کاربر اعلام می‌کند چنین فردی پیدا نشده است.

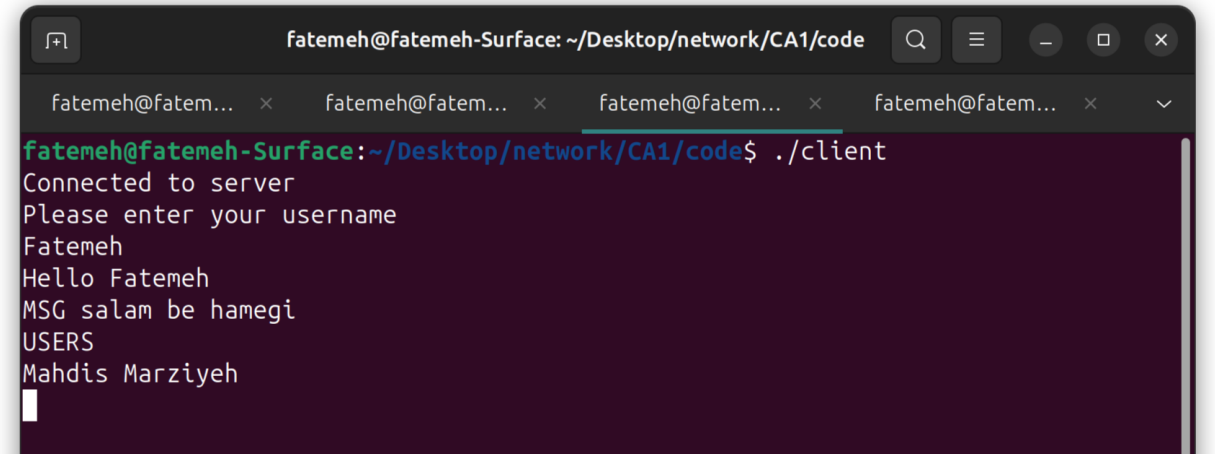
```
void handle_pm(int client_socket, string username, string message)
{
    string pv_message = "PRIVATE-MESSAGE from " + clients[client_socket] + " : " + message;
    for (auto &c : clients)
    {
        if (c.second == username)
        {
            message_to_client(c.first, pv_message);
            return;
        }
    }
    message_to_client(client_socket, "User not found");
}
```




fatemeh@fatemeh-Surface: ~/Desktop/network/CA1/code
fatemeh@fatem...
fatemeh@fatem...
fatemeh@fatem...
fatemeh@fatem...
fatemeh@fatemeh-Surface:~/Desktop/network/CA1/code$ g++ client.cpp -o client
fatemeh@fatemeh-Surface:~/Desktop/network/CA1/code$ ./client
Connected to server
Please enter your username
Mahdis
Hello Mahdis
PUBLIC-MESSAGE from Fatemeh : salam be hamegi
PRIVATE-MESSAGE from Marziyeh : payam baraye mahdis

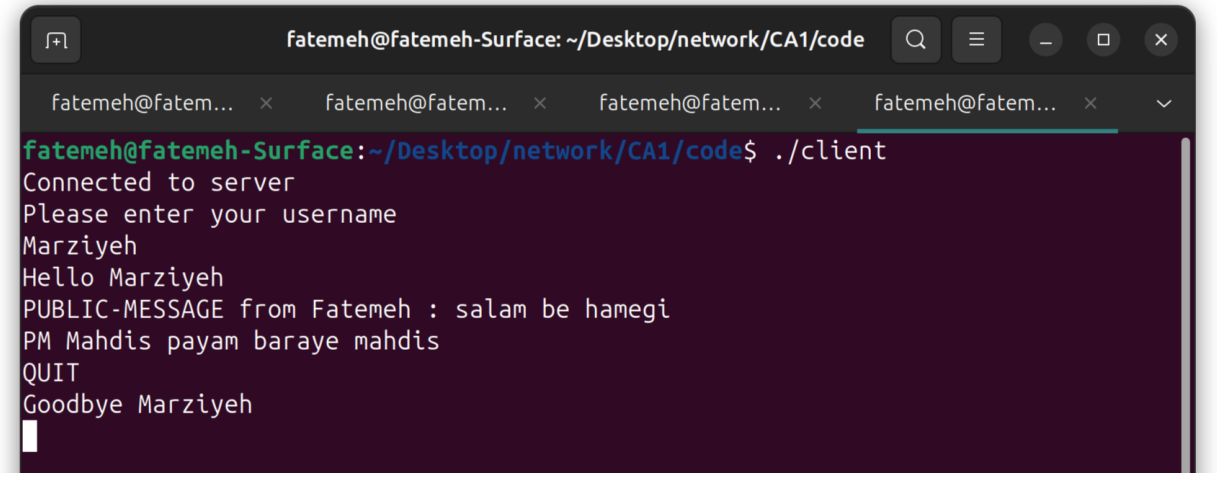
USERS

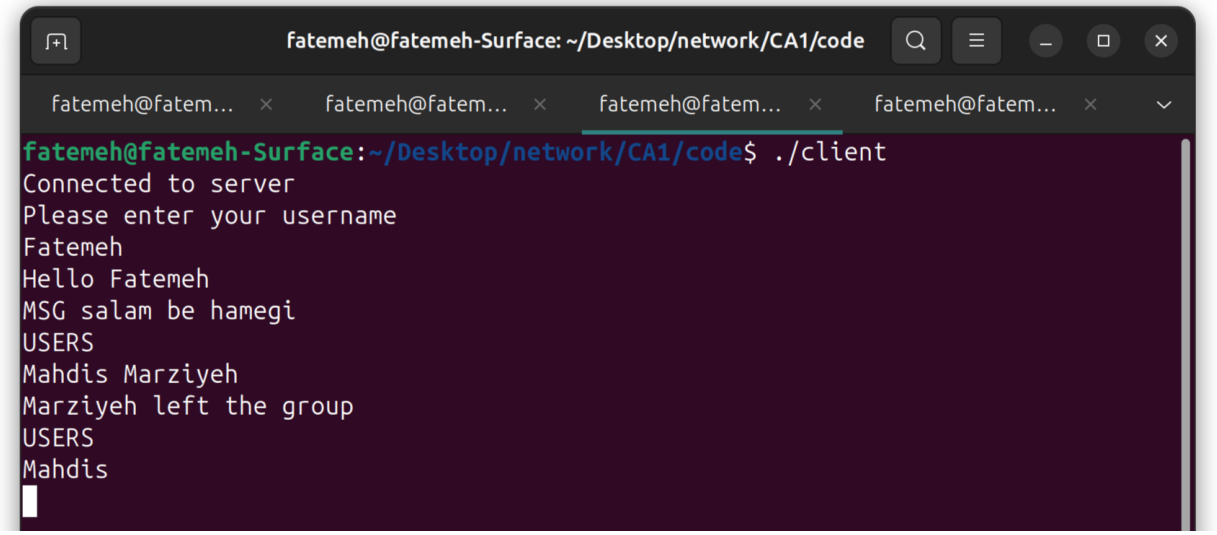
اسامی افراد آنلاین به جز خودش نشان داده می‌شود.



اگر یکی از کاربران QUIT کند، دیگر نمایش داده نمی‌شود. طبق تابع زیر از بین کلاینت‌ها بین افرادی که هنوز وجود دارند و غیر از کاربری هستند که از این دستور استفاده کرده است اسامی سایرین با تابع کمکی message_to_client برای کلاینت ارسال می‌شود.

```
void handle_users(int client_socket)
{
    string online_users = "";
    for (auto &c : clients)
    {
        if (c.first != client_socket && !c.second.empty())
        {
            online_users += c.second + " ";
        }
    }
    message_to_client(client_socket, online_users);
}
```

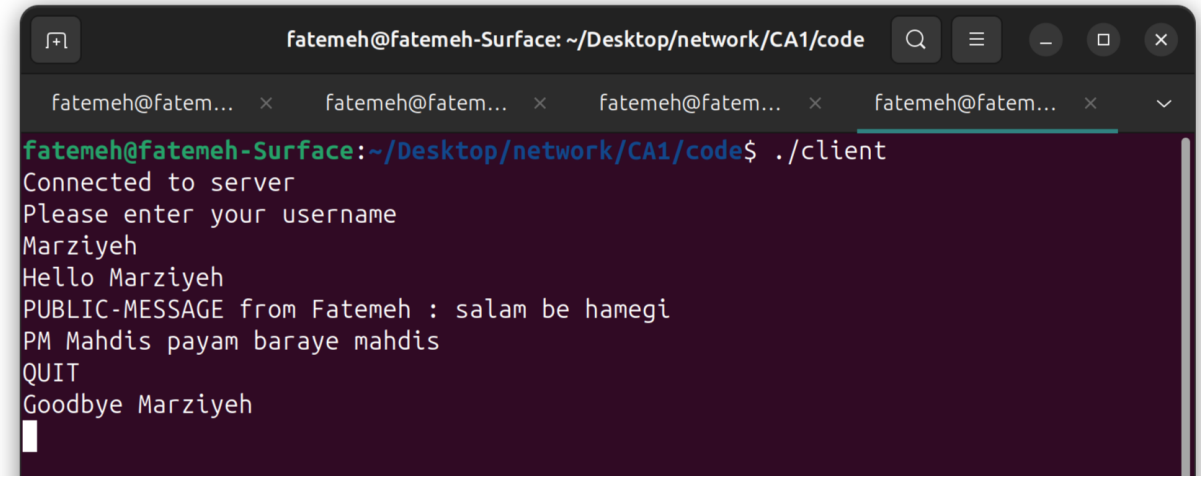

fatemeh@fatemeh-Surface: ~/Desktop/network/CA1/code
fatemeh@fatem...
fatemeh@fatem...
fatemeh@fatem...
fatemeh@fatem...
fatemeh@fatemeh-Surface:~/Desktop/network/CA1/code$ ./client
Connected to server
Please enter your username
Marziyeh
Hello Marziyeh
PUBLIC-MESSAGE from Fatemeh : salam be hamegi
PM Mahdis payam baraye mahdis
QUIT
Goodbye Marziyeh

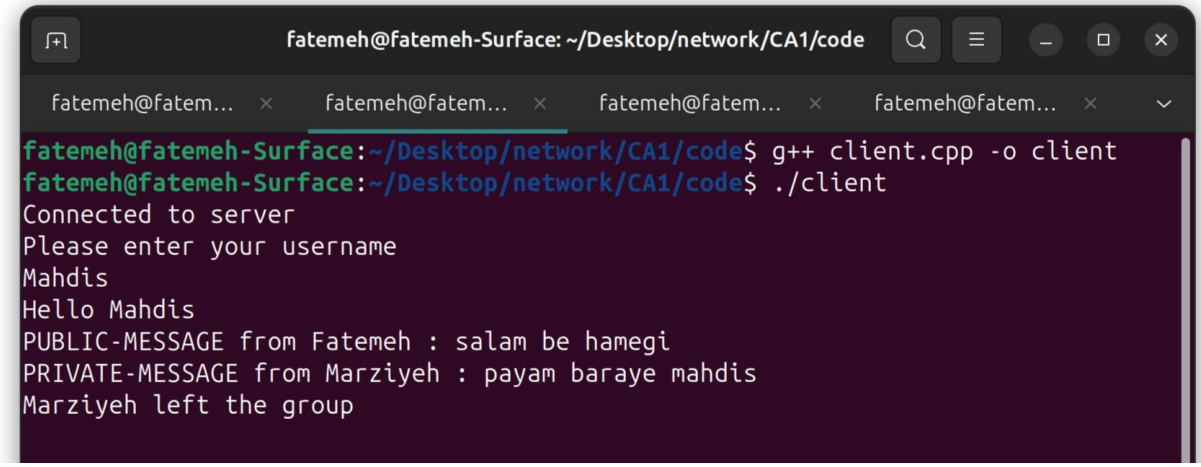

fatemeh@fatemeh-Surface: ~/Desktop/network/CA1/code
fatemeh@fatem...
fatemeh@fatem...
fatemeh@fatem...
fatemeh@fatem...
fatemeh@fatemeh-Surface:~/Desktop/network/CA1/code$ ./client
Connected to server
Please enter your username
Fatemeh
Hello Fatemeh
MSG salam be hamegi
USERS
Mahdis Marziyeh
Marziyeh left the group
USERS
Mahdis

QUIT

اگر کاربری QUIT کند، به او پیام خداحافظی نشان داده می‌شود و به بقیه کاربران اطلاع داده می‌شود. این فرایند در تابع زیر نمایش داده شده است که با تابع broadcast برای سایرین پیام رفتن فرد و با تابع message_to_client برای خود فرد پیام خداحافظی ارسال می‌شود و سپس از map پاک می‌شود.

```
void handle_quit(int client_socket)
{
    message_to_client(client_socket, "Goodbye " + clients[client_socket]);
    broadcast(clients[client_socket] + " left the group", client_socket);
    clients.erase(client_socket);
}
```





PUT file_path

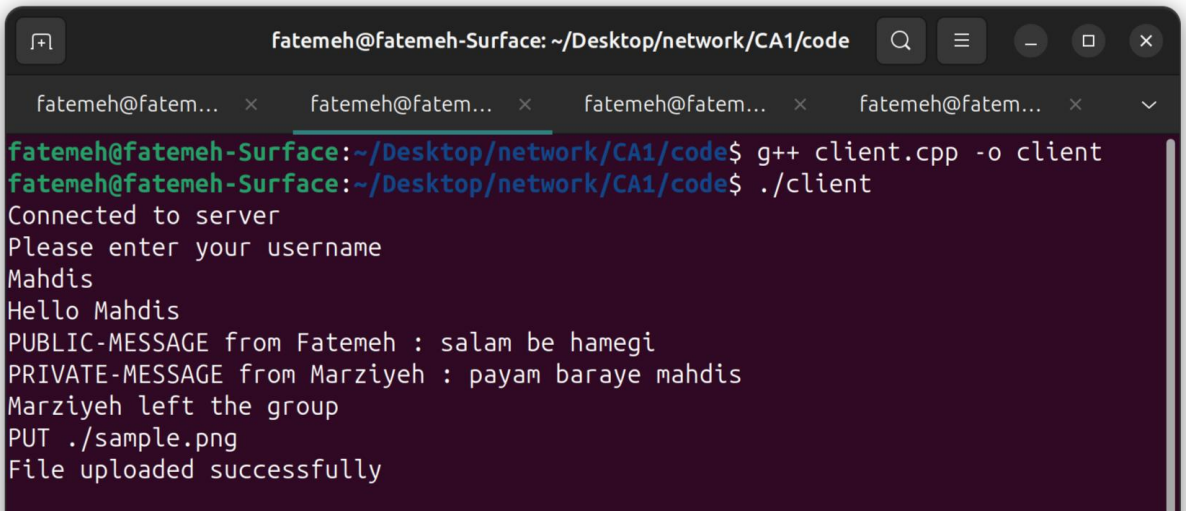
سرور در ابتدای اجرای برنامه یک directory به نام server_files تشکیل می‌دهد. هر زمان فایلی بخواد روی سرور قرار بگیرد، این فایل در این directory کپی می‌شود.

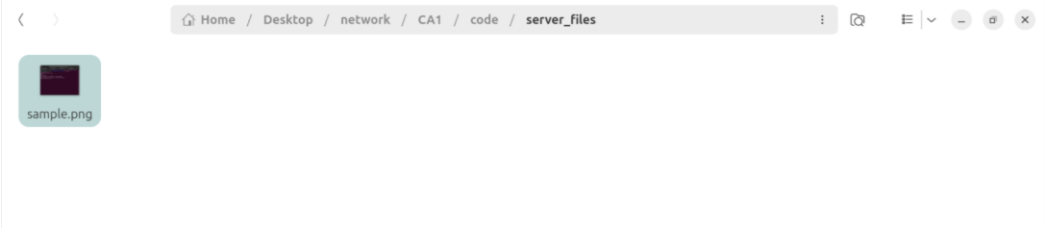
مسیر file_path بر اساس محل اجرای client است. پس از آپلود فایل، پیام موفقیت به کاربر نشان داده می‌شود.

در کد زیر فایل از مسیر source در مسیر destination کپی می‌شود. file_name نام فایل را مشخص می‌کند که فایل در سرور چه نامی بگیرد. این نام همان نامی است که در مسیر source وجود دارد.

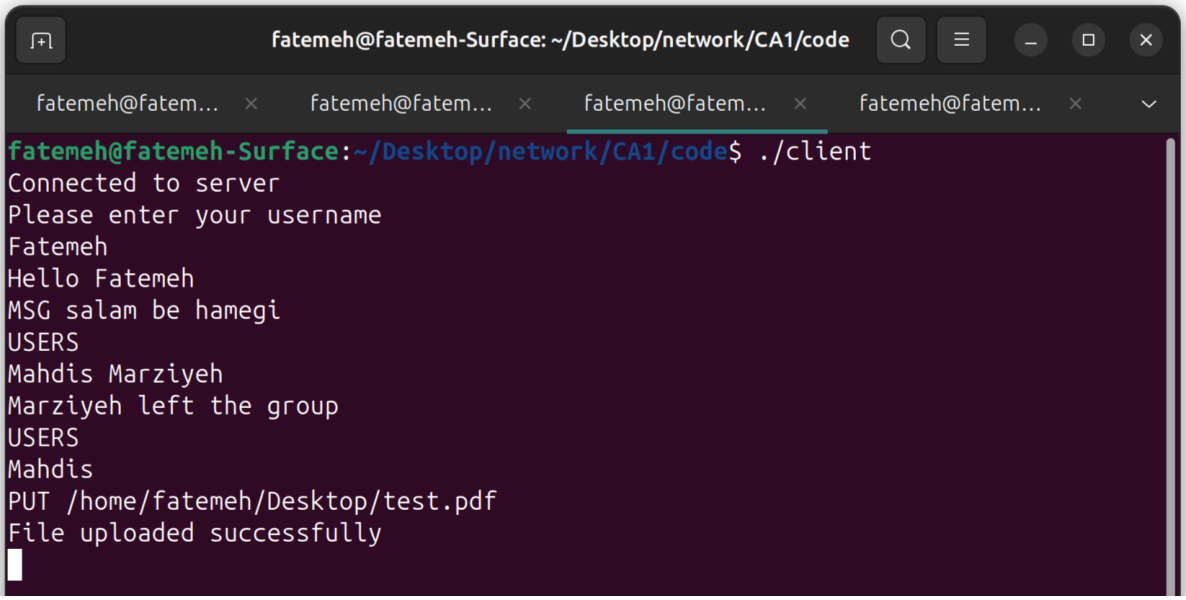
در صورتی که کپی با خطا مواجه شود (exception رخ دهد)، پیغام مناسب به کار نمایش داده می‌شود.

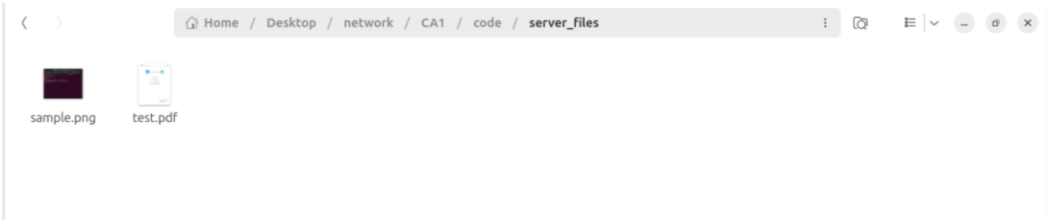
```
void handle_put(int client_socket, string file_path)
{
    filesystem::path source = file_path;
    string file_name = source.filename();
    filesystem::path destination = SERVER_FILES_DIR + file_name;
    try
    {
        std::filesystem::copy_file(
            source,
            destination,
            std::filesystem::copy_options::overwrite_existing);
        message_to_client(client_socket, "File uploaded successfully");
    }
    catch (const std::filesystem::filesystem_error &e)
    {
        message_to_client(client_socket, "Failed to upload the file");
    }
}
```





برای ادامه توضیحات یک فایل دیگر به نام test.pdf را نیز آپلود می‌کنیم.

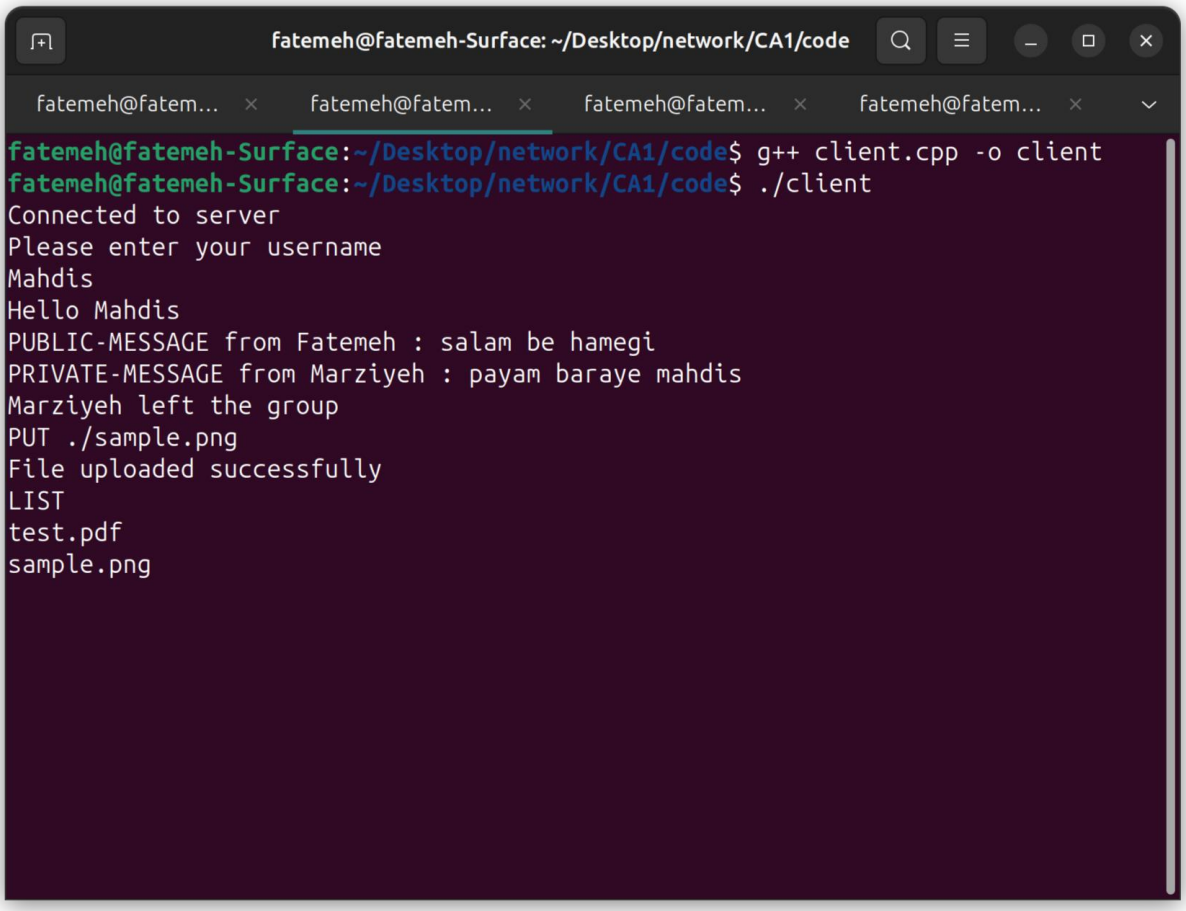




LIST

این دستور نام تمام فایل‌های موجود را نشان می‌دهد.
برای این کار در کد زیر، تمام فایل‌های داخل SERVER_FILES_DIR بررسی می‌شوند و نام تمام آن‌ها نمایش داده می‌شود. با توجه به این قابلیت، نیازی به ذخیره نام‌ها در هنگام آپلود روی سرور نیست.

```
void handle_list(int client_socket)
{
    string all_files = "";
    for (const auto &entry : filesystem::directory_iterator(SERVER_FILES_DIR))
    {
        string file_name = entry.path().filename().string();
        all_files += file_name + "\n";
    }
    all_files = all_files.substr(0, all_files.size() - 1);
    message_to_client(client_socket, all_files);
}
```



GET file_name file_destination

این دستور فایل را از سرور در مسیر مورد نظر کاربر کپی می‌کند. مسیر file_destination نسبت به محل اجرای client است.
در کد زیر فایل از مسیر source در مسیر destination کپی می‌شود. file_name نام فایل را مشخص می‌کند که توسط client داده شده است. این نام همان نامی است که در مسیر source وجود دارد. در صورتی که کپی با خطا مواجه شود (exception رخ دهد)، پیغام مناسب به کار نمایش داده می‌شود.

```
void handle_get(int client_socket, string file_name, string destination_path)
{
    filesystem::path source = SERVER_FILES_DIR + file_name;
    filesystem::path destination = destination_path + file_name;
    try
    {
        std::filesystem::copy_file(
            source,
            destination,
            std::filesystem::copy_options::overwrite_existing);
        message_to_client(client_socket, "File downloaded successfully");
    }
    catch (const std::filesystem::filesystem_error &e)
    {
        message_to_client(client_socket, "Failed to download the file");
    }
}
```

